

方形件组批优化问题

一、问题概述

本问题包含两个子问题：

- 子问题1 (排样优化)**：给定同一材质的方形件集合，在满足"齐头切" (Guillotine cut) 约束和3阶段精确排样约束下，最小化使用的原片数量。
- 子问题2 (订单组批+排样优化)**：对多材质、多订单的方形件集合进行组批，每个批次内相同材质的件才能共用同一原片，同时满足批次容量约束，最终最小化总原片用量。

二、符号说明

2.1 集合与索引

符号	说明
I	产品项 (item) 集合, 索引 $i \in I$
J	批次集合, 索引 $j \in J$
M	材质集合, 索引 $m \in M$
O	订单集合, 索引 $o \in O$
I_o	属于订单 o 的产品项子集
I_m	材质为 m 的产品项子集
K	原片集合, 索引 $k \in K$

2.2 参数

符号	说明
L	原片长度, $L = 2440$ mm
W	原片宽度, $W = 1220$ mm
l_i	产品项 i 的长度 (mm)
w_i	产品项 i 的宽度 (mm)
n_i	产品项 i 的需求数量
$N_{\max}$	单批次产品项数上限, $N_{\max} = 1000$
$A_{\max}$	单批次面积上限, $A_{\max} = 250$ m ²
a_i	产品项 i 的面积, $a_i = l_i \times w_i$

2.3 决策变量

符号	类型	说明
x_{ij}	0-1变量	产品项 i 是否分配到批次 j
y_{jk}	0-1变量	批次 j 是否使用原片 k
z_{ik}	0-1变量	产品项 i 是否排布在原片 k 上
p_{ik}^x	连续变量	产品项 i 在原片 k 上的x坐标（左下角）
p_{ik}^y	连续变量	产品项 i 在原片 k 上的y坐标（左下角）
o_i	0-1变量	产品项 i 是否旋转90°（长宽互换）
u_k	0-1变量	原片 k 是否被使用

三、子问题1：排样优化模型

3.1 约束条件

约束(1)：目标函数

最小化使用的原片总数。

$$\min \sum_{k \in K} u_k$$

约束(2)：需求满足约束

每个产品项必须被完整排布到原片上，需求量 n_i 必须全部满足。

$$\sum_{k \in K} z_{ik} = n_i, \quad \forall i \in I$$

约束(3)-(4)：旋转定义

l'_i 和 w'_i 为考虑旋转后的有效长宽。当 $o_i = 0$ 时不旋转， $o_i = 1$ 时旋转90°（长宽互换）。

$$l'_i = l_i(1 - o_i) + w_i \cdot o_i, \quad \forall i \in I$$

$$w'_i = w_i(1 - o_i) + l_i \cdot o_i, \quad \forall i \in I$$

约束(5)-(6)：原片边界约束

产品项不能超出原片边界。采用大M法：当产品项 i 排布在原片 k 上（ $z_{ik} = 1$ ）时，约束退化为 $p_{ik}^x + l'_i \leq L$ 和 $p_{ik}^y + w'_i \leq W$ ；当 $z_{ik} = 0$ 时，大M使约束自动松弛，坐标变量不受限制。其中 M 为足够大的常数（可取 $M = \max(L, W)$ ）。

$$p_{ik}^x + l'_i \leq L + M(1 - z_{ik}), \quad \forall i \in I, k \in K$$

$$p_{ik}^y + w'_i \leq W + M(1 - z_{ik}), \quad \forall i \in I, k \in K$$

约束(7)-(11)：不重叠约束

对于同一原片 k 上的任意两个产品项 i_1, i_2 ，至少满足以下四个方向之一的不重叠条件：

$$p_{i_1 k}^x + l'_{i_1} \leq p_{i_2 k}^x \vee p_{i_2 k}^x + l'_{i_2} \leq p_{i_1 k}^x \vee p_{i_1 k}^y + w'_{i_1} \leq p_{i_2 k}^y \vee p_{i_2 k}^y + w'_{i_2} \leq p_{i_1 k}^y$$

使用大M法将析取约束线性化，其中 M 为足够大的常数， $\alpha_{i_1 i_2 k}^r \in \{0, 1\}$ 为辅助0-1变量，约束(11)确保至少一个方向的不重叠条件被激活。

$$p_{i_1 k}^x + l'_{i_1} \leq p_{i_2 k}^x + M(1 - \alpha_{i_1 i_2 k}^1), \quad \forall i_1 < i_2, k \in K$$

$$p_{i_2 k}^x + l'_{i_2} \leq p_{i_1 k}^x + M(1 - \alpha_{i_1 i_2 k}^2), \quad \forall i_1 < i_2, k \in K$$

$$p_{i_1 k}^y + w'_{i_1} \leq p_{i_2 k}^y + M(1 - \alpha_{i_1 i_2 k}^3), \quad \forall i_1 < i_2, k \in K$$

$$p_{i_2 k}^y + w'_{i_2} \leq p_{i_1 k}^y + M(1 - \alpha_{i_1 i_2 k}^4), \quad \forall i_1 < i_2, k \in K$$

$$\alpha_{i_1 i_2 k}^1 + \alpha_{i_1 i_2 k}^2 + \alpha_{i_1 i_2 k}^3 + \alpha_{i_1 i_2 k}^4 \geq 1, \quad \forall i_1 < i_2, k \in K$$

约束(12)-(14): 齐头切 (Guillotine Cut) 约束

排样方案必须满足3阶段齐头切约束：

- **第1阶段**：原片沿水平方向切割为若干条带 (Stripe)，每条带宽度为 h_s (s 为条带索引)，约束(12)保证条带总宽度不超过原片宽度
- **第2阶段**：每条带沿垂直方向切割为若干栈 (Stack)，约束(13)保证同一栈内产品项y方向宽度相同
- **第3阶段**：每栈沿水平方向切割为产品项 (Item)，约束(14)保证同一栈内产品项长度之和不超过原片长度

$$\sum_s h_s \leq W, \quad h_s > 0$$

$$w'_{i_1} = w'_{i_2}, \quad \forall i_1, i_2 \in \text{Stack}_t$$

$$\sum_{i \in \text{Stack}_t} l'_i \leq L$$

约束(15): 变量关联约束

确保产品项只能排布到已使用的原片上。

$$z_{ik} \leq u_k, \quad \forall i \in I, k \in K$$

约束(16): 坐标非负约束

$$p_{ik}^x \geq 0, \quad p_{ik}^y \geq 0, \quad \forall i \in I, k \in K$$

约束(17): 变量域约束

$$z_{ik}, u_k, o_i \in \{0, 1\}, \quad \alpha_{i_1 i_2 k}^r \in \{0, 1\}$$

3.2 数学规划标准形式

$$\min \sum_{k \in K} u_k$$

$$\begin{aligned}
& \sum_{k \in K} z_{ik} = n_i, & \forall i \in I & (2) \\
& l'_i = l_i(1 - o_i) + w_i \cdot o_i, & \forall i \in I & (3) \\
& w'_i = w_i(1 - o_i) + l_i \cdot o_i, & \forall i \in I & (4) \\
& p_{ik}^x + l'_i \leq L + M(1 - z_{ik}), & \forall i \in I, k \in K & (5) \\
& p_{ik}^y + w'_i \leq W + M(1 - z_{ik}), & \forall i \in I, k \in K & (6) \\
& p_{i_1k}^x + l'_{i_1} \leq p_{i_2k}^x + M(1 - \alpha_{i_1i_2k}^1), & \forall i_1 < i_2, k \in K & (7) \\
& p_{i_2k}^x + l'_{i_2} \leq p_{i_1k}^x + M(1 - \alpha_{i_1i_2k}^2), & \forall i_1 < i_2, k \in K & (8) \\
& p_{i_1k}^y + w'_{i_1} \leq p_{i_2k}^y + M(1 - \alpha_{i_1i_2k}^3), & \forall i_1 < i_2, k \in K & (9) \\
& p_{i_2k}^y + w'_{i_2} \leq p_{i_1k}^y + M(1 - \alpha_{i_1i_2k}^4), & \forall i_1 < i_2, k \in K & (10) \\
& \alpha_{i_1i_2k}^1 + \alpha_{i_1i_2k}^2 + \alpha_{i_1i_2k}^3 + \alpha_{i_1i_2k}^4 \geq 1, & \forall i_1 < i_2, k \in K & (11) \\
& \sum_s h_s \leq W, \quad h_s > 0 & & (12) \\
& w'_{i_1} = w'_{i_2}, & \forall i_1, i_2 \in \text{Stack}_t & (13) \\
& \sum_{i \in \text{Stack}_t} l'_i \leq L & & (14) \\
& z_{ik} \leq u_k, & \forall i \in I, k \in K & (15) \\
& p_{ik}^x \geq 0, p_{ik}^y \geq 0, & \forall i \in I, k \in K & (16) \\
& z_{ik}, u_k, o_i \in \{0, 1\}, \alpha_{i_1i_2k}^r \in \{0, 1\} & & (17)
\end{aligned}$$

s.t.

四、子问题2：订单组批+排样优化模型

4.1 约束条件

在子问题1全部约束（约束(2)-(17)）的基础上，增加以下约束：

约束(18)：目标函数

最小化所有批次使用的原片总数。

$$\min \sum_{j \in J} \sum_{k \in K_j} u_{jk}$$

约束(19)：订单完整分配约束

每个产品项当且仅当分配到一个批次，确保所有产品项都被处理且不重复分配。

$$\sum_{j \in J} x_{ij} = 1, \quad \forall i \in I$$

约束(20)：材质内订单完整性约束

同一订单中相同材质的产品项必须分配到同一批次。由于同一原片只能排布相同材质的产品项（约束(23)），算法按材质分别组批，同订单不同材质的项可分到不同批次，但同材质的项必须保持完整，确保同材质的订单项不被拆分。

$$x_{i_1j} = x_{i_2j}, \quad \forall i_1, i_2 \in I_o \cap I_m, \forall j \in J, \forall m \in M$$

约束(21)：批次产品项数约束

每个批次的产品项总数不超过 $N_{\max} = 1000$ 。

$$\sum_{i \in I} x_{ij} \leq N_{\max}, \quad \forall j \in J$$

约束(22): 批次面积约束

每个批次的产品项面积总和不超过 $A_{\max} = 250 \text{ m}^2$ 。

$$\sum_{i \in I} a_i \cdot x_{ij} \leq A_{\max}, \quad \forall j \in J$$

约束(23): 材质共用约束

同一原片上只能排布相同材质的产品项。若两个产品项材质不同，则不能排布在同一原片上。

$$z_{i_1 k} + z_{i_2 k} \leq 1, \quad \text{if } m_{i_1} \neq m_{i_2}, \forall i_1, i_2 \in I, k \in K$$

约束(24): 变量域约束

x_{ij} 为0-1决策变量，表示产品项 i 是否分配到批次 j 。

$$x_{ij} \in \{0, 1\}, \quad \forall i \in I, j \in J$$

4.2 数学规划标准形式

$$\min \sum_{j \in J} \sum_{k \in K_j} u_{jk}$$

$$\text{s.t.} \begin{cases} \text{约束(2)-(17)} \text{ (继承子问题1全部约束)} \\ \sum_{j \in J} x_{ij} = 1, & \forall i \in I & (19) \\ x_{i_1 j} = x_{i_2 j}, & \forall i_1, i_2 \in I_o \cap I_m, \forall j \in J, \forall m \in M & (20) \\ \sum_{i \in I} x_{ij} \leq N_{\max}, & \forall j \in J & (21) \\ \sum_{i \in I} a_i \cdot x_{ij} \leq A_{\max}, & \forall j \in J & (22) \\ z_{i_1 k} + z_{i_2 k} \leq 1, & \text{if } m_{i_1} \neq m_{i_2}, \forall i_1, i_2 \in I, k \in K & (23) \\ x_{ij} \in \{0, 1\}, & \forall i \in I, j \in J & (24) \end{cases}$$

五、板材利用率

$$\text{利用率} = \frac{\sum_{i \in I} l_i \times w_i \times n_i}{\sum_{k \in K} L \times W \times u_k}$$

六、模型复杂度分析

该问题属于强NP-hard问题。子问题1的决策变量数和约束数随产品项数呈二次增长（不重叠约束），子问题2进一步引入批次分配变量，问题规模更大。

对于数据集A（750项/文件），精确求解已极具挑战；对于数据集B（18000-28000项/文件），必须采用启发式算法。

求解策略

由于该问题属于强NP-hard问题，精确求解不可行，我们设计了多层次启发式算法框架，包含**排样优化算法**和**订单组批算法**两大模块。

七、排样优化算法

7.0 数据模型

排样算法使用以下核心数据结构（定义在 `solver/models.py`）：

`PlacedItem`：已放置的产品项

属性	类型	说明
<code>item_id</code>	int	产品ID
<code>material</code>	str	材质
<code>x</code>	float	左下角x坐标 (mm)
<code>y</code>	float	左下角y坐标 (mm)
<code>length</code>	float	x方向长度 (mm)
<code>width</code>	float	y方向长度 (mm)
<code>rotated</code>	bool	是否旋转90°
<code>order</code>	str	订单号

`CuttingBoard`：一块原片的排样结果

属性	类型	说明
<code>board_id</code>	int	原片序号
<code>material</code>	str	材质
<code>plate_length</code>	float	原片长度 (mm)
<code>plate_width</code>	float	原片宽度 (mm)
<code>placed_items</code>	List[PlacedItem]	已放置产品项列表
<code>used_area</code>	float (property)	已使用面积
<code>total_area</code>	float (property)	原片总面积
<code>utilization</code>	float (property)	利用率

7.1 算法总体框架

排样算法采用**两层类继承结构**：

- **基础类** `GuillotineCutPacker`：实现3阶段齐头切排样的核心逻辑（朝向决策、宽度聚类、栈构建、原片装载），尝试3组容差 × 3种朝向 = 9种组合择优
- **增强类** `EnhancedGuillotinePacker`：继承基础类，扩展为4组容差 × 3种朝向 = 12种组合 + 2种 Shelf算法 + 20次随机扰动 × 2种朝向 = 40种组合，共54种方案比较择优，并增加原片合并后处理

核心思路为：

1. **朝向决策**：确定每个产品项在原片上的放置朝向（是否旋转90°）
2. **宽度聚类**：将相近宽度的产品项归并为同一组，减少栈数量
3. **栈构建**：在同一宽度组内，用BFD策略将产品项装入栈

4. **原片装载**：用全局最优适配算法将栈装入原片
5. **多策略比较**：尝试多种参数组合，选最优结果
6. **随机扰动**：多次随机打乱输入顺序，取最优
7. **后处理合并**：尝试将稀疏原片的产品项合并到其他原片

7.2 朝向决策 (`_orient_items`)

每个产品项有两种可能的朝向：

朝向	x方向长度 el	y方向宽度 ew	旋转标志
A (不旋转)	l_i	w_i	False
B (旋转90°)	w_i	l_i	True

朝向选择策略：

- `width_first`：优先短边作为y方向 (ew)，有利于栈宽度一致。当 $l \geq w$ 时选择朝向A ($ew = w$)，否则选择朝向B ($ew = l$)
- `length_first`：优先长边作为y方向，可能产生更深的栈。当 $l < w$ 时选择朝向A ($ew = w$)，否则选择朝向B ($ew = l$)
- `hybrid`：当前实现与 `width_first` 策略一致 ($prefer_a = (l \geq w)$)，保留接口以供后续扩展

朝向选择算法：

对每个产品项 $item(l, w)$ ：

1. 计算两种朝向：
 - 朝向A: $el = l, ew = w, rotated = \text{False}$
 - 朝向B: $el = w, ew = l, rotated = \text{True}$
2. 检查可行性: $fits_a = (el_a \leq L \wedge ew_a \leq W)$, $fits_b = (el_b \leq L \wedge ew_b \leq W)$
3. 根据策略确定优先朝向：
 - `width_first`: $prefer_a = (l \geq w)$
 - `length_first`: $prefer_a = (l < w)$
 - `hybrid`: $prefer_a = (l \geq w)$
4. 选择朝向（优先使用策略偏好的可行朝向）：
 - 若 $prefer_a \wedge fits_a \rightarrow$ 选朝向A
 - 否则若 $\neg prefer_a \wedge fits_b \rightarrow$ 选朝向B
 - 否则若 $fits_a \rightarrow$ 选朝向A
 - 否则若 $fits_b \rightarrow$ 选朝向B
 - 否则 $\rightarrow$ 选溢出较小的朝向

关键约束： $el \leq L$ 且 $ew \leq W$ 。若优先朝向超出原片范围，自动切换到另一朝向。

7.3 宽度聚类归并 (`_build_stacks`)

7.3.1 动机

3阶段齐头切要求同一栈内产品项y方向宽度相同。若按精确宽度分组，大量窄栈会浪费垂直空间。宽度聚类允许相近宽度的项共享同一栈，以少量垂直空间浪费换取更少的栈数和更高的原片利用率。

7.3.2 贪心聚类算法

输入：朝向后的产品项列表（按ew升序排列）

算法步骤：

1. 初始化聚类列表 `clusters = []`

2. 对每个产品项 `item`（按 `ew` 升序）：

a. 计算容差 `tol = max(cluster_w × tol_pct, tol_min)`

b. 遍历已有聚类，找到第一个满足 `ew ≤ cluster_w + tol` 的聚类

c. 若找到：将 `item` 加入该聚类，更新 `cluster_w = max(cluster_w, ew)`

d. 若未找到：创建新聚类 `{items: [item], cluster_w: ew}`

3. 对每个聚类内的项按 `el` 降序排列

4. 对每个聚类用BFD策略装栈
- 容差参数：
- 增强版排样器（`EnhancedGuillotinePacker`）尝试4组容差参数：
- | 编号 | tol_pct | tol_min | 适用场景 |
|----|---------|---------|-------------|
| 1 | 0.5% | 1mm | 精确匹配，浪费最小 |
| 2 | 1% | 2mm | 小容差，平衡精度与归并 |
| 3 | 2% | 5mm | 中等容差，适度归并 |
| 4 | 3% | 8mm | 较大容差，强归并 |
- 基础版排样器（`GuillotineCutPacker`）尝试3组容差参数：
- | 编号 | tol_pct | tol_min | 适用场景 |
|----|---------|---------|---------|
| 1 | 2% | 5mm | 中等容差 |
| 2 | 5% | 10mm | 较大容差 |
| 3 | 10% | 20mm | 大容差，强归并 |
- 7.3.3 BFD装栈
- 对每个宽度聚类组，使用Best Fit Decreasing策略将产品项装入栈：
1. 将聚类内项按 `el` 降序排列

2. 维护活跃栈列表 `active_stacks = [(remaining_length, [items])]`

3. 对每个项 `item`：

a. 在所有 `remaining_length ≥ item.el` 的栈中，找剩余长度最小的（最佳匹配）

b. 若找到：将 `item` 加入该栈，更新 `remaining_length`

c. 若未找到：创建新栈，`remaining_length = L - item.el`

4. 每个栈的宽度 = `cluster_w`（聚类宽度，而非项的最大宽度）

输出：栈列表 `[[{items, stack_w, stack_l}]]`，其中 `stack_w` 为y方向宽度，`stack_l` 为x方向总长度。

7.4 全局最优适配装原片 (`_pack_stacks_to_boards`)

7.4.1 多排序策略

将栈装入原片前，尝试5种排序策略，对每种排序独立求解，选原片数最少的：

策略名	排序键	说明
<code>w_desc_l_desc</code>	$(-stack_w, -stack_l)$	宽度优先降序
<code>l_desc_w_desc</code>	$(-stack_l, -stack_w)$	长度优先降序
<code>area_desc</code>	$-(stack_w \times stack_l)$	面积降序
<code>w_desc_l_asc</code>	$(-stack_w, stack_l)$	宽降序+长升序
<code>w_asc_l_desc</code>	$(stack_w, -stack_l)$	宽升序+长降序

7.4.2 全局最优适配算法 (`_pack_stacks_to_boards_single`)

对排好序的栈列表，逐个放入原片，每次选择**全局最优位置**：

1. 初始化原片列表 `boards_data = []`
2. 对每个栈 `sinfo`（按排序顺序）：
 - a. `best_score = ∞`
 - b. 遍历所有已有原片 `bdata`：
 - i. 遍历该原片的所有已有条带 `stripe`：
 - 若 $w_{stack} \leq h_s$ 且 $l_{used} + l_{stack} \leq L$ ：
$$score = (h_s - w_{stack}) \times l_{stack} + (L - l_{used} - l_{stack}) \times \min(w_{stack}, h_s - w_{stack})$$

若 `score < best_score`：更新最佳位置
 - ii. 尝试在该原片创建新条带：
 - 若 $y_{used} + w_{stack} \leq W$ ：
$$score = (W - y_{used} - w_{stack}) \times L \times 0.1$$
（新条带惩罚系数）
若 `score < best_score`：更新最佳位置
 - c. 若找到最佳位置：将栈放入对应原片的对应条带
 - d. 若未找到：创建新原片，新条带

评分函数设计：

$$score = \underbrace{(h_s - w_{stack}) \times l_{stack}}_{\text{高度浪费}} + \underbrace{(L - l_{used} - l_{stack}) \times \min(w_{stack}, h_s - w_{stack})}_{\text{长度浪费预估}}$$

评分越小表示浪费越少，优先选择。新条带有额外惩罚系数0.1，避免过早开新条带。

7.5 Shelf排样算法变体

除条带-栈结构外，还实现了两种Shelf排样算法作为补充策略：

7.5.1 按面积降序Shelf算法 (`_pack_shelf_area_desc`)

采用 `width_first` 朝向策略，产品项按面积 $el \times ew$ 降序排列。

1. 朝向决策：`_orient_items(items, 'width_first')`
2. 按面积降序排列：`sorted(oriented, key =  $-(el \times ew)$ )`
3. 初始化：`boards = []`, `shelves = []`, `current_y = 0`

4. 对每个产品项 item (按面积降序) :
 - a. 在已有 shelf 中找最佳匹配:
 - 遍历所有 shelf, 找满足 $ew \leq \text{shelf.height}$ 且 $\text{used_l} + el \leq L$ 的 shelf
 - 选择高度浪费 ($\text{shelf.height} - ew$) 最小的 shelf
 - b. 若找到最佳 shelf: 将 item 放入该 shelf, 更新 used_l
 - c. 若未找到:
 - i. 若 $\text{current_y} + ew \leq W$: 创建新 shelf, $\text{current_y} += ew$
 - ii. 否则: 将当前 shelves 封装为原片, 创建新原片和新 shelf
5. 将剩余 shelves 封装为原片
6. 返回 boards

7.5.2 按长边降序Shelf算法 (`_pack_shelf_length_desc`)

采用 `width_first` 朝向策略, 产品项按长边 el 降序排列, 其余逻辑同面积降序版本。

1. 朝向决策: `_orient_items(items, 'width_first')`
2. 按长边降序排列: `sorted(oriented, key = -el)`
3. 其余步骤同 `_pack_shelf_area_desc` 的步骤3-6

7.6 随机扰动优化

在确定性策略基础上, 增加随机扰动以跳出局部最优:

1. 设定随机种子 $\text{seed} = 42$ (可复现)
2. 重复20次:
 - a. 随机打乱产品项顺序
 - b. 对每种朝向策略 (`width_first`, `length_first`) :
 - o 朝向决策 $\rightarrow$ 宽度聚类($1\%/2\text{mm}$) $\rightarrow$ 栈构建 $\rightarrow$ 装原片
 - c. 若原片数更少则更新最优解

7.7 原片合并后处理 (`_consolidate_boards`)

尝试将利用率低的原片上的产品项移到其他原片的剩余空间:

1. 若原片数 ≤ 1 , 直接返回
2. 循环 (直到无改进) :
 - a. 计算每个原片的利用率 $\text{util} = \text{used_area}/\text{plate_area}$
 - b. 按利用率升序排列
 - c. 对利用率 $< 85\%$ 的原片 (源原片 `src_board`) :
 - i. 收集其所有产品项为 `items_to_place`
 - ii. 遍历其他原片 (目标原片 `dst_board`) :
 - 计算 $y_{\text{used}} = \max(\text{item.y} + \text{item.width})$ (目标原片已用 y 空间)
 - $\text{remaining_y} = W - y_{\text{used}}$
 - 对 `items_to_place` 中每个项:
 - 若 $ew \leq \text{remaining_y}$ 且 $el \leq L$:
 - 放置到目标原片 ($x = 0, y = y_{\text{used}}$), 更新 y_{used} 和 remaining_y
 - 否则: 加入 `still_remaining` 列表
 - `items_to_place = still_remaining`
 - iii. 若 `items_to_place` 为空 (所有项已移走) :
 - 移除源原片, 标记 `improved = True`, 跳出当前循环
 - d. 重新编号原片
3. 返回合并后的原片列表

7.8 基础版排样器总流程 (GuillotineCutPacker.pack)

输入: 产品项 DataFrame, 材质名

输出: 排样结果列表 (CuttingBoard)

1. 展开 item_num 为单独项 (_expand_items)
2. 3种容差 $\times$ 3种朝向 = 9 种组合
对每种组合: _orient_items $\rightarrow$ _build_stacks $\rightarrow$ _pack_stacks_to_boards
保留原片数最少的解
3. 返回最优解

总计比较 $3 \times 3 = 9$ 种排样方案, 选原片数最少的。

7.9 增强版排样器总流程 (EnhancedGuillotinePacker.pack)

输入: 产品项 DataFrame, 材质名

输出: 排样结果列表 (CuttingBoard)

1. 展开 item_num 为单独项 (_expand_items)
2. 策略1: 4种容差 $\times$ 3种朝向 = 12 种组合
对每种组合: _orient_items $\rightarrow$ _build_stacks $\rightarrow$ _pack_stacks_to_boards
保留原片数最少的解
3. 策略2: Shelf 面积降序算法 (_pack_shelf_area_desc)
4. 策略3: Shelf 长边降序算法 (_pack_shelf_length_desc)
5. 策略4: 20次随机扰动 $\times$ 2种朝向 = 40 种组合
每次随机打乱项序 $\rightarrow$ _orient_items $\rightarrow$ _build_stacks(1%/2mm) $\rightarrow$ _pack_stacks_to_boards
6. 后处理: _consolidate_boards (合并利用率 $< 85\%$ 的稀疏原片)
7. 返回最优解

总计比较 $12 + 2 + 40 = 54$ 种排样方案, 选原片数最少的。

八、订单组批算法

8.1 问题分析

子问题2在排样优化基础上增加组批决策, 需满足:

- 每份订单在同一材质组内当且仅当出现在一个批次中 (跨材质可分批)
- 每个批次产品项总数 $\leq N_{\max} = 1000$
- 每个批次产品项面积总和 $\leq A_{\max} = 250 \text{ m}^2$
- 同一原片只能排布相同材质的产品项

8.2 贪心组批算法 (OrderBatcher.batch_orders)

核心思路: 按材质分组, 每种材质内按订单面积降序贪心装批。

1. 统计每个订单的产品项总数和总面积 (跨所有材质合计)
2. 按材质对订单分组 (一个订单可能包含多种材质, 按材质拆分, 同一订单可出现在多个材质组中)

3. 对每种材质：
 - a. 将该材质的订单按总面积降序排列
 - b. 初始化当前批次: $\text{count} = 0, \text{area} = 0$
 - c. 对每个订单（面积降序）：
 - o 若 $\text{count} + \text{order_count} \leq 1000$ 且 $\text{area} + \text{order_area} \leq 250\text{m}^2$ （ order_count 和 order_area 为该订单跨所有材质的总量，非仅当前材质部分）：
加入当前批次
 - o 否则：保存当前批次，创建新批次
4. 返回所有批次

8.3 组批+排样联合流程

对每个数据集文件：

1. 读取 CSV，加载产品项
2. 调用 `OrderBatcher.batch_orders` 进行组批
3. 对每个批次：
 - a. 按材质分组
 - b. 对每种材质调用 `EnhancedGuillotinePacker.pack` 排样
4. 统计总原片数和利用率
5. 输出组批方案 CSV

九、实验结果

9.1 子问题1（数据集A） — 排样优化

数据集	产品项数	材质数	原片数	利用率
dataA1	757	1	93	90.01%
dataA2	734	1	93	89.17%
dataA3	828	1	93	90.15%
dataA4	813	1	92	89.25%
dataA5	753	1	93	90.18%
合计	3885	—	464	89.75%

9.2 子问题2（数据集B） — 订单组批+排样优化

数据集	产品项数	材质数	订单数	批次数	原片数	利用率
dataB1	26811	130	546	251	18326	74.42%
dataB2	17952	146	403	214	12312	72.11%
dataB3	18028	160	410	220	11999	73.02%
dataB4	18526	142	381	221	12790	74.22%
dataB5	27901	192	604	304	19998	70.17%
合计	109218	—	2344	1210	75425	72.66%

9.3 坐标正确性验证

所有输出CSV均通过坐标验证：

- **边界约束**：0越界 ($x + l \leq 2440, y + w \leq 1220$)
- **不重叠约束**：0重叠（同一原片内任意两项不重叠）
- 子问题2验证需按（批次序号，原片材质，原片序号）三元组分组

9.4 算法效果分析

子问题1利用率较高的原因：

- 单材质场景，所有产品项可自由组合
- 宽度聚类归并减少了栈数量
- 全局最优适配减少了原片间浪费
- 多策略比较+随机扰动找到了更好的解

子问题2利用率相对较低的原因：

- 多材质约束：同一原片只能放相同材质，限制了组合自由度
- 批次容量约束：每批 ≤ 1000 项/250m²，可能导致小批次
- 材质内订单完整性约束：同一订单同材质的项必须同批，跨材质可分批但增加组批复杂度
- 组批与排样的耦合：当前采用先组批后排样的两阶段策略，未考虑排样效果对组批的反馈

十、输出格式

10.1 子问题1输出（cut_program_dataA*.csv）

列名	说明
原片材质	原片材质名称
原片序号	原片编号（从0开始）
产品id	产品项编号
产品x坐标	产品项左下角x坐标（mm）
产品y坐标	产品项左下角y坐标（mm）
产品x方向长度	产品项x方向尺寸（mm）
产品y方向长度	产品项y方向尺寸（mm）

10.2 子问题2输出（sum_order_dataB*.csv）

列名	说明
批次数号	批次编号（从0开始）
原片材质	原片材质名称
原片序号	原片编号（同一材质内从0开始）
产品id	产品项编号
产品x坐标	产品项左下角x坐标（mm）
产品y坐标	产品项左下角y坐标（mm）
产品x方向长度	产品项x方向尺寸（mm）
产品y方向长度	产品项y方向尺寸（mm）